Lab 7: Fuzzing for Input Validation Bugs

Project: Hospital Triage Priority (processor.py)

Sunidhi Shekar

PES1UG24AM410

Objective

To apply property-based fuzz testing with the Hypothesis library to find input validation bugs in processor.py (parse_vitals_summary, compute_patient_bmi, assign_ward_bed), and then harden the code so that no malformed input can crash it.

Step 1: Install Dependencies

Installed the required libraries inside the project virtual environment:

pip install hypothesis

pip install pytest

Step 2: Run the Buggy Application

Command: python processor.py

2a. Valid input

With well-formed input (130,92,37.5 / 70/1.75 / ICU-05) all three functions work correctly.

```
(.venv) PS C:\dev\SE_LAB_PES1UG24AM410_Sunidhi_Shekar_LAB\PES1UG24AM410_SELAB7_47> python processor.py
==== Running parse_vitals_summary ====
Enter vitals (HR,SpO2,Temp): 130,92,37.5
Summary: Vitals: HR=130.0, SpO2=92.0% - CRITICAL

==== Running compute_patient_bmi ====
Enter weight/height (e.g. 70/1.75): 70/1.75
BMI: 22.86

==== Running assign_ward_bed ====
Enter allocation (e.g. ICU-05): ICU-05
Assignment: Intensive Care Unit Bed #5
(.venv) PS C:\dev\SE_LAB_PES1UG24AM410_Sunidhi_Shekar_LAB\PES1UG24AM410_SELAB7_47>
```

Figure 1: Buggy processor.py on valid input

2b. Malformed input (crash)

Entering only two vitals (130,92) crashes the program with IndexError: list index out of range, because the code assumes three comma-separated values exist.

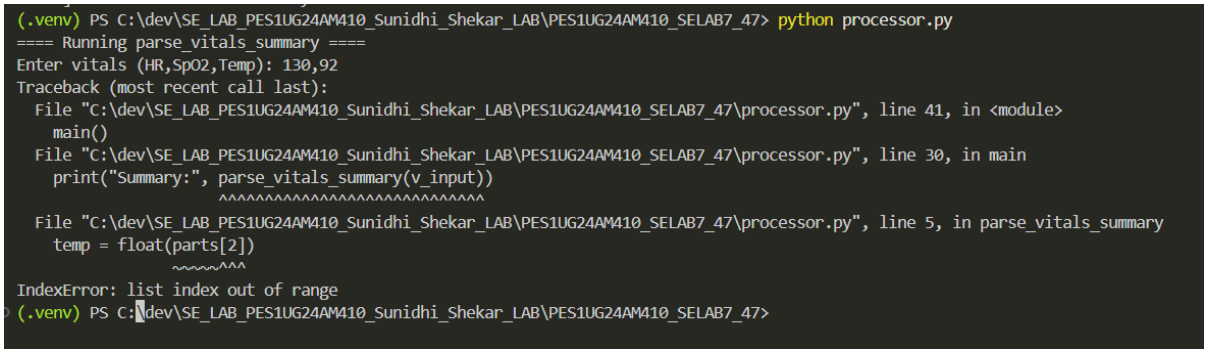


Figure 2: Buggy processor.py crashing on input "130,92"

Step 3: Complete test_processor.py with Hypothesis

Each of the three tests uses @given(st.text() | st.none()) to generate random strings and None. The function under test is wrapped in try/except; any exception fails the test with a message showing the offending input. Each test also asserts the return type (string, or float/string for the BMI function).

Step 4: Run Tests Against the Buggy Code

Command: pytest test_processor.py

All 3 tests fail. Hypothesis discovered, among others, these failing inputs:

- parse_vitals_summary: '0' (IndexError), '' (ValueError), None (AttributeError)
- compute_patient_bmi: '' and '/' (ValueError), None (AttributeError), plus 2 further distinct failures
- assign_ward_bed: '-' and '' (ValueError), None (AttributeError)

```
(.venv) PS C:\dev\SE_LAB_PES1UG24AM410_Sunidhi_Shekar_LAB\PES1UG24AM410_SELAB7_47> pytest test_processor.py
========================= test session starts =========================
platform win32 -- Python 3.12.4, pytest-9.2.0, pluggy-1.7.0
rootdir: C:\dev\SE_LAB_PES1UG24AM410_Sunidhi_Shekar_LAB\PES1UG24AM410_SELAB7_47
plugins: hypothesis-6.170.15
collected 3 items

test_processor.py FFF                                                [100%]

============================== FAILURES ===============================
_____________ test_parse_vitals_summary_no_crash ______________
  + Exception Group Traceback (most recent call last):
  |   File "C:\dev\SE_LAB_PES1UG24AM410_Sunidhi_Shekar_LAB\PES1UG24AM410_SELAB7_47\test_processor.py", line 6, in test_parse_vitals_summary_no_crash
  |     def test_parse_vitals_summary_no_crash(s):
  |                    ^^^
  | ExceptionGroup: Hypothesis found 3 distinct failures. (3 sub-exceptions)
  +-+---------------- 1 ----------------
    | Traceback (most recent call last):
    |   File "C:\dev\SE_LAB_PES1UG24AM410_Sunidhi_Shekar_LAB\PES1UG24AM410_SELAB7_47\test_processor.py", line 8, in test_parse_vitals_summary_no_crash
    |     result = parse_vitals_summary(s)
    |              ^^^^^^^^^^^^^^^^^^^^^^
    |   File "C:\dev\SE_LAB_PES1UG24AM410_Sunidhi_Shekar_LAB\PES1UG24AM410_SELAB7_47\processor.py", line 4, in parse_vitals_summary
    |     spo2 = float(parts[1])
    |            ~~~~~^^^
    | IndexError: list index out of range
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "C:\dev\SE_LAB_PES1UG24AM410_Sunidhi_Shekar_LAB\PES1UG24AM410_SELAB7_47\test_processor.py", line 10, in test_parse_vitals_summary_no_crash
    |     assert False, f"parse_vitals_summary crashed with: {e!r} on input {s!r}"
    | AssertionError: parse_vitals_summary crashed with: IndexError('list index out of range') on input '0'
    | assert False
    | Failing test case: test_parse_vitals_summary_no_crash(
    |     s='0',
    | )
    +---------------- 2 ----------------
    | Traceback (most recent call last):
    |   File "C:\dev\SE_LAB_PES1UG24AM410_Sunidhi_Shekar_LAB\PES1UG24AM410_SELAB7_47\test_processor.py", line 8, in test_parse_vitals_summary_no_crash
    |     result = parse_vitals_summary(s)
    |              ^^^^^^^^^^^^^^^^^^^^^^
    |   File "C:\dev\SE_LAB_PES1UG24AM410_Sunidhi_Shekar_LAB\PES1UG24AM410_SELAB7_47\processor.py", line 3, in parse_vitals_summary
    |     hr = float(parts[0])
    |          ^^^^^^^^^^^^^^
    | ValueError: could not convert string to float: ''
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "C:\dev\SE_LAB_PES1UG24AM410_Sunidhi_Shekar_LAB\PES1UG24AM410_SELAB7_47\test_processor.py", line 10, in test_parse_vitals_summary_no_crash
    |     assert False, f"parse_vitals_summary crashed with: {e!r} on input {s!r}"
    | AssertionError: parse_vitals_summary crashed with: ValueError("could not convert string to float: ''") on input ''
    | assert False
    | Failing test case: test_parse_vitals_summary_no_crash(
    |     s='',  # or any other generated value
    | )
    +---------------- 3 ----------------
    | Traceback (most recent call last):
    |   File "C:\dev\SE_LAB_PES1UG24AM410_Sunidhi_Shekar_LAB\PES1UG24AM410_SELAB7_47\test_processor.py", line 8, in test_parse_vitals_summary_no_crash
    |     result = parse_vitals_summary(s)
```

Figure 3: pytest run (FFF) with the first Hypothesis failures

```
    | ValueError: not enough values to unpack (expected 2, got 1)
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "C:\dev\SE_LAB_PES1UG24AM410_Sunidhi_Shekar_LAB\PES1UG24AM410_SELAB7_47\test_processor.py", line 30, in test_assign_ward_bed_safe
    |     assert False, f"assign_ward_bed crashed with: {e!r} on input {s!r}"
    | AssertionError: assign_ward_bed crashed with: ValueError('not enough values to unpack (expected 2, got 1)') on input ''
    | assert False
    | Failing test case: test_assign_ward_bed_safe(
    |     s='',  # or any other generated value
    | )
    +---------------- 3 ----------------
    | Traceback (most recent call last):
    |   File "C:\dev\SE_LAB_PES1UG24AM410_Sunidhi_Shekar_LAB\PES1UG24AM410_SELAB7_47\test_processor.py", line 28, in test_assign_ward_bed_safe
    |     result = assign_ward_bed(s)
    |              ^^^^^^^^^^^^^^^^^
    |   File "C:\dev\SE_LAB_PES1UG24AM410_Sunidhi_Shekar_LAB\PES1UG24AM410_SELAB7_47\processor.py", line 17, in assign_ward_bed
    |     ward, bed_str = allocation_code.strip().split('-')
    |                     ^^^^^^^^^^^^^^^^^^^^
    | AttributeError: 'NoneType' object has no attribute 'strip'
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "C:\dev\SE_LAB_PES1UG24AM410_Sunidhi_Shekar_LAB\PES1UG24AM410_SELAB7_47\test_processor.py", line 30, in test_assign_ward_bed_safe
    |     assert False, f"assign_ward_bed crashed with: {e!r} on input {s!r}"
    | AssertionError: assign_ward_bed crashed with: AttributeError("'NoneType' object has no attribute 'strip'") on input None
    | assert False
    | Failing test case: test_assign_ward_bed_safe(
    |     s=None,  # or any other generated value
    | )
    +------------------------------------
=================== short test summary info ===================
FAILED test_processor.py::test_parse_vitals_summary_no_crash - ExceptionGroup: Hypothesis found 3 distinct failures. (3 sub-exce...
FAILED test_processor.py::test_compute_patient_bmi_safe - ExceptionGroup: Hypothesis found 5 distinct failures. (5 sub-exce...
FAILED test_processor.py::test_assign_ward_bed_safe - ExceptionGroup: Hypothesis found 3 distinct failures. (3 sub-exce...
====================== 3 failed in 5.90s ======================
(.venv) PS C:\dev\SE_LAB_PES1UG24AM410_Sunidhi_Shekar_LAB\PES1UG24AM410_SELAB7_47>
```

Figure 4: pytest failure summary: 3 failed in 5.90s

Step 5: Fix the Buggy Application

Changes made to processor.py to harden all three functions:

- Type checking with isinstance(x, str) so None and non-string inputs are rejected.
- Structure checks on the split result (exactly 3 parts for vitals, 2 for BMI and ward-bed) to prevent IndexError and unpacking ValueError.
- A safe helper, _to_finite_float(), that wraps float() in try/except and rejects NaN and infinity.
- Range checks: SpO2 0-100, realistic temperature and heart rate, positive weight and height (prevents ZeroDivisionError and negative BMI).
- Ward name validated against the known wards before the dictionary lookup (prevents KeyError).
- Bed number must be ASCII digits, at least 1, and of limited length (rejects empty strings, signs, Unicode digits and huge values).
- Invalid input now returns a clear "Invalid input: ..." message instead of raising an exception.

Command: python processor.py

5a. Valid input

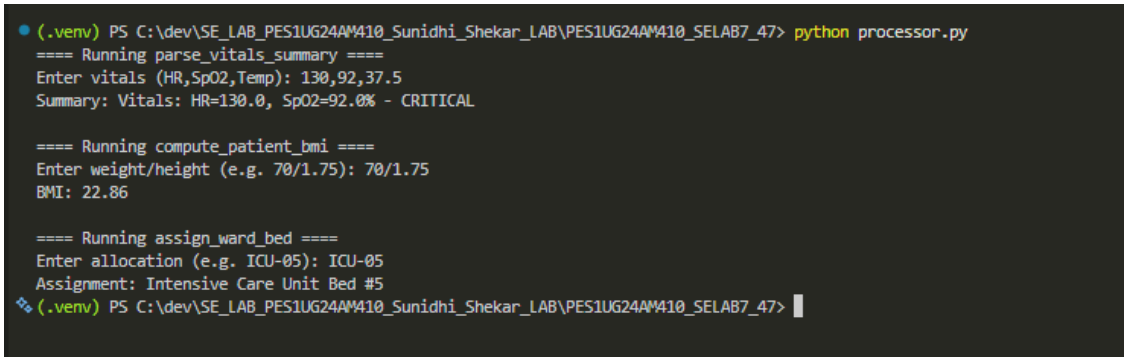


Figure 5: Fixed processor.py on valid input (unchanged behaviour)

5b. Invalid input

The same inputs that crashed earlier (130,92), plus a zero height (70/0) and an unknown ward (XYZ-9), are now handled gracefully.

```
(.venv) PS C:\dev\SE_LAB_PES1UG24AM410_Sunidhi_Shekar_LAB\PES1UG24AM410_SELAB7_47> python processor.py
==== Running parse_vitals_summary ====
Enter vitals (HR,SpO2,Temp): 130,92
Summary: Invalid input: expected exactly 3 values 'HR,SpO2,Temp'

==== Running compute_patient_bmi ====
Enter weight/height (e.g. 70/1.75): 70/0
BMI: Invalid input: weight and height must be positive

==== Running assign_ward_bed ====
Enter allocation (e.g. ICU-05): XYZ-9
Assignment: Invalid input: unknown ward 'XYZ'
(.venv) PS C:\dev\SE_LAB_PES1UG24AM410_Sunidhi_Shekar_LAB\PES1UG24AM410_SELAB7_47>
```

Figure 6: Fixed processor.py handling invalid input without crashing

Step 6: Re-run Tests on the Fixed Code

Command: pytest test_processor.py -v

All 3 fuzz tests now pass, confirming that none of the functions crash on any of the inputs generated by Hypothesis.

```
(.venv) PS C:\dev\SE_LAB_PES1UG24AM410_Sunidhi_Shekar_LAB\PES1UG24AM410_SELAB7_47> pytest test_processor.py -v
============================================= test session starts =============================================
platform win32 -- Python 3.12.4, pytest-9.2.0, pluggy-1.7.0 -- C:\dev\SE_LAB_PES1UG24AM410_Sunidhi_Shekar_LAB\PES1UG24AM410_SELAB7_47\.venv\Scripts\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\dev\SE_LAB_PES1UG24AM410_Sunidhi_Shekar_LAB\PES1UG24AM410_SELAB7_47
plugins: hypothesis-6.170.15
collected 3 items

test_processor.py::test_parse_vitals_summary_no_crash PASSED                                          [ 33%]
test_processor.py::test_compute_patient_bmi_safe PASSED                                               [ 66%]
test_processor.py::test_assign_ward_bed_safe PASSED                                                   [100%]

============================================== 3 passed in 0.99s ==============================================
(.venv) PS C:\dev\SE_LAB_PES1UG24AM410_Sunidhi_Shekar_LAB\PES1UG24AM410_SELAB7_47>
```

Figure 7: pytest on fixed code: 3 passed

Conclusion

Fuzz testing exposed edge cases (empty strings, None, missing separators, division by zero, unknown ward codes) that a few hand-written unit tests would likely have missed. After adding defensive validation, all three functions handle arbitrary input safely and the Hypothesis tests pass.

Files Submitted

- processor.py (fixed application script)
- test_processor.py (completed Hypothesis tests)
- PES1UG24AM410_SELAB7_report.pdf